

專案名稱：簡易購物商城

第000組

作者：000

指導老師：張庭禎老師

簡易購物商城大綱

1. 專案介紹
2. 使用案例 (Use Case)
3. 系統功能說明
4. 資料庫模型圖
5. 前端程式架構說明
6. 後端程式架構說明

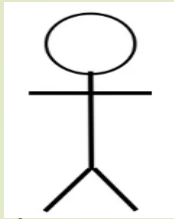
專案介紹

- 專案為線上商城系統，包含帳戶登入、商品瀏覽、購物車管理及訂單查詢。
- 技術：前端使用HTML + jQuery+ Bootstrap 框架
後端Spring Boot + Data JPA 進行資料互動(登入與取得產品列表)。
- 使用場景：模擬線上購物流程，小型電商平台。

未來使用案例（ Use Case ）擴充角色

角色	權限
一般使用者(目前)	登入系統、查看商品、加入購物車、送出訂單、查詢自己訂單
管理者（可擴充）	未於此前端實作，未來可新增刪除商品、查看所有訂單狀態等功能

使用案例 (Use Case)



角色:使用者

使用案例名稱	說明
使用者登入	使用者輸入帳號密碼，透過 <code>/api/user/login</code> 取得 JWT Token 並登入。
瀏覽商品列表	使用者點擊「商品」頁籤時，透過 <code>/api/products</code> 載入商品資料並顯示。
加入購物車	點選商品卡片的「加入購物車」按鈕，即將該商品與指定數量加入購物車陣列。
檢視購物車	顯示購物車清單與總價，可刪除項目。
送出訂單	登入後按下「送出訂單」，將購物車內容與使用者帳號組成 JSON 傳送至 <code>/api/orders</code> 。
查詢歷史訂單	登入後切換至「訂單」頁籤，從 <code>/api/orders/{username}</code> 載入訂單列表。
查詢訂單明細	點選訂單後，顯示該筆訂單資料與明細，呼叫 <code>/api/orders/orderid/{id}</code> 及 <code>/api/items/{id}</code> 。

系統功能說明

1. 頁面區域：登入、產品列表、訂單、購物車
2. 主要功能：
 - 帳戶登入功能
 - 產品顯示與加入購物車
 - 訂單管理
 - 顯示購物車並送出訂單

1. 登入系統

- ◊ 首頁預設會顯示登入畫面。

- ◊ 請輸入您的帳號密碼，例如：

帳號：admin

密碼：1234

- ◊ 按下「登入」按鈕後，右上角會顯示登入成功訊息（例如：歡迎 admin）。

☒ 提示：未登入無法送出訂單，也無法查詢個人訂單。

我的商城 帳戶登入 產品列表 訂單 購物車 未登入

帳戶登入

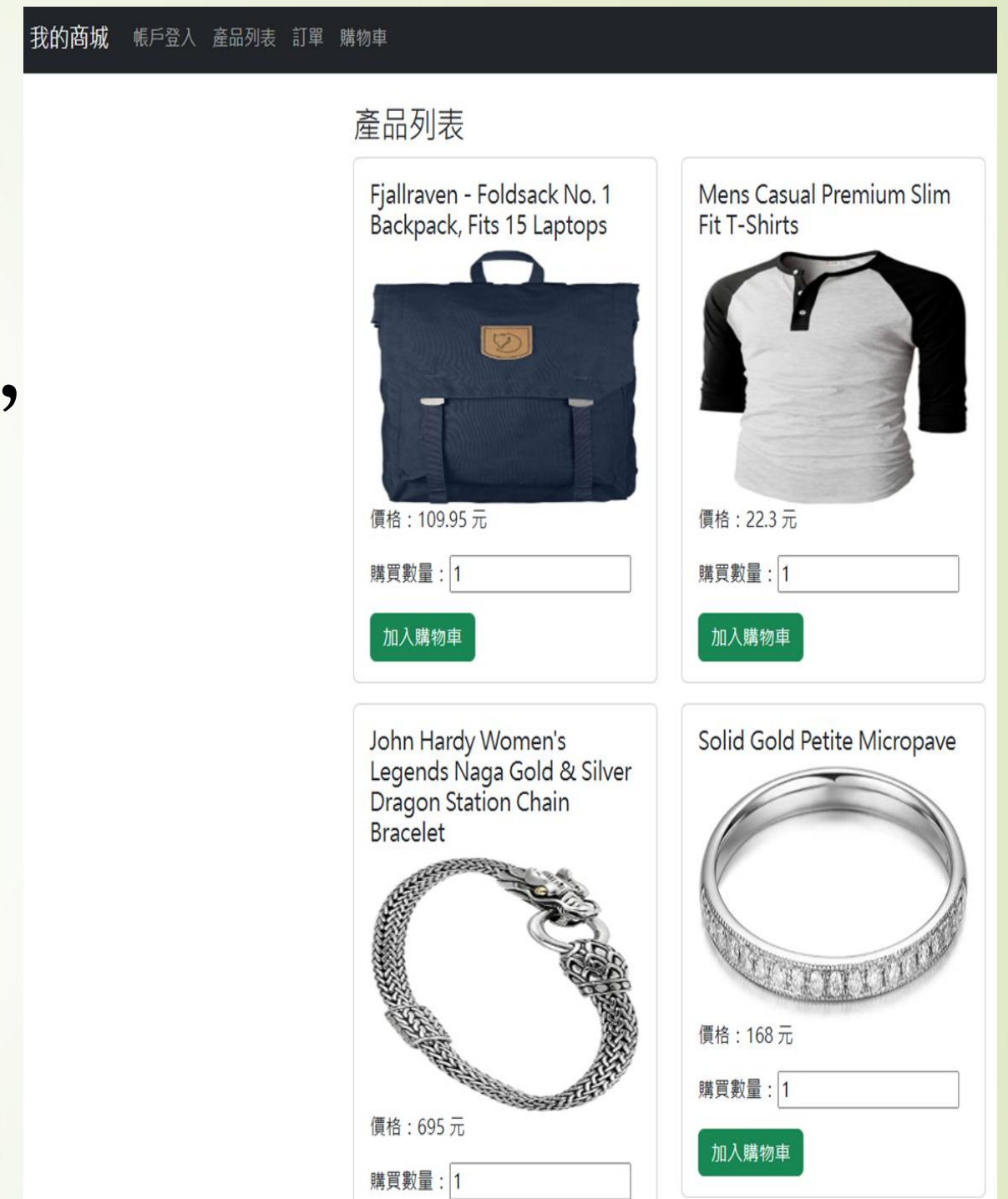
admin

1234

登入

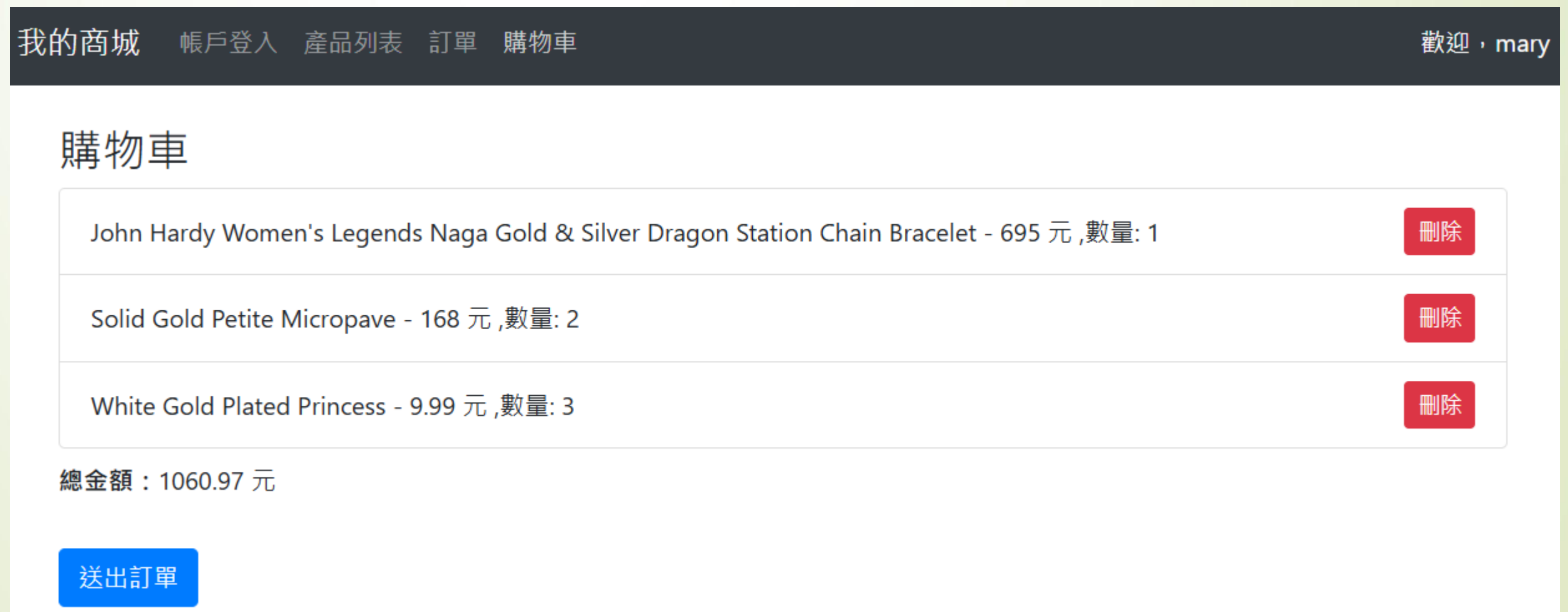
2. 瀏覽產品

- ◊ 點選上方導覽列的「產品列表」。
- ◊ 系統會列出目前可購買的商品。
- ◊ 每個商品旁邊有「加入購物車」按鈕，
可以將想要的商品放入購物車。
- ☑ 提示：商品資料由後端提供，每次開啟頁面會自動重新載入最新商品。



3. 管理購物車

- ◊ 點選「購物車」進入購物車畫面。
- ◊ 可以看到已經加入的商品清單，包括商品名稱、價格。
- ◊ 可以刪除購物車中的商品。
- ◊ 底部會顯示總金額。



4. 送出訂單

- ◇ 確認購物車內容無誤後，按下「送出訂單」按鈕。
 - ◇ 系統會將您的購物車資料打包成一筆新訂單送到後端儲存。
 - ◇ 成功送出後，系統會清空購物車並顯示提示訊息。
- ☒ **注意：**必須登入後才能送出訂單！

5. 查詢歷史訂單

- 點選「訂單」進入訂單管理畫面。
 - ◊ 系統會列出目前帳號所有歷史訂單。
 - ◊ 點擊訂單旁的「查看商品明細」按鈕，可以展開該筆訂單內的所有商品內容。
- ☒ **提示：** 訂單列表會即時更新，送出新訂單後請重新整理或返回訂單頁面查詢。

我的商城

帳戶登入

產品列表

訂單

購物車

歡迎，mary

訂單管理

訂單編號

7

訂單用戶

mary

訂單時間

2025-04-28T11:10:08.22615

商品明細

產品編號

5

產品名稱

John Hardy Women's Legends Naga Gold & Silver Dragon Station Chain Bracelet

產品價格

695

數量

1

6

Solid Gold Petite Micropave

168

2

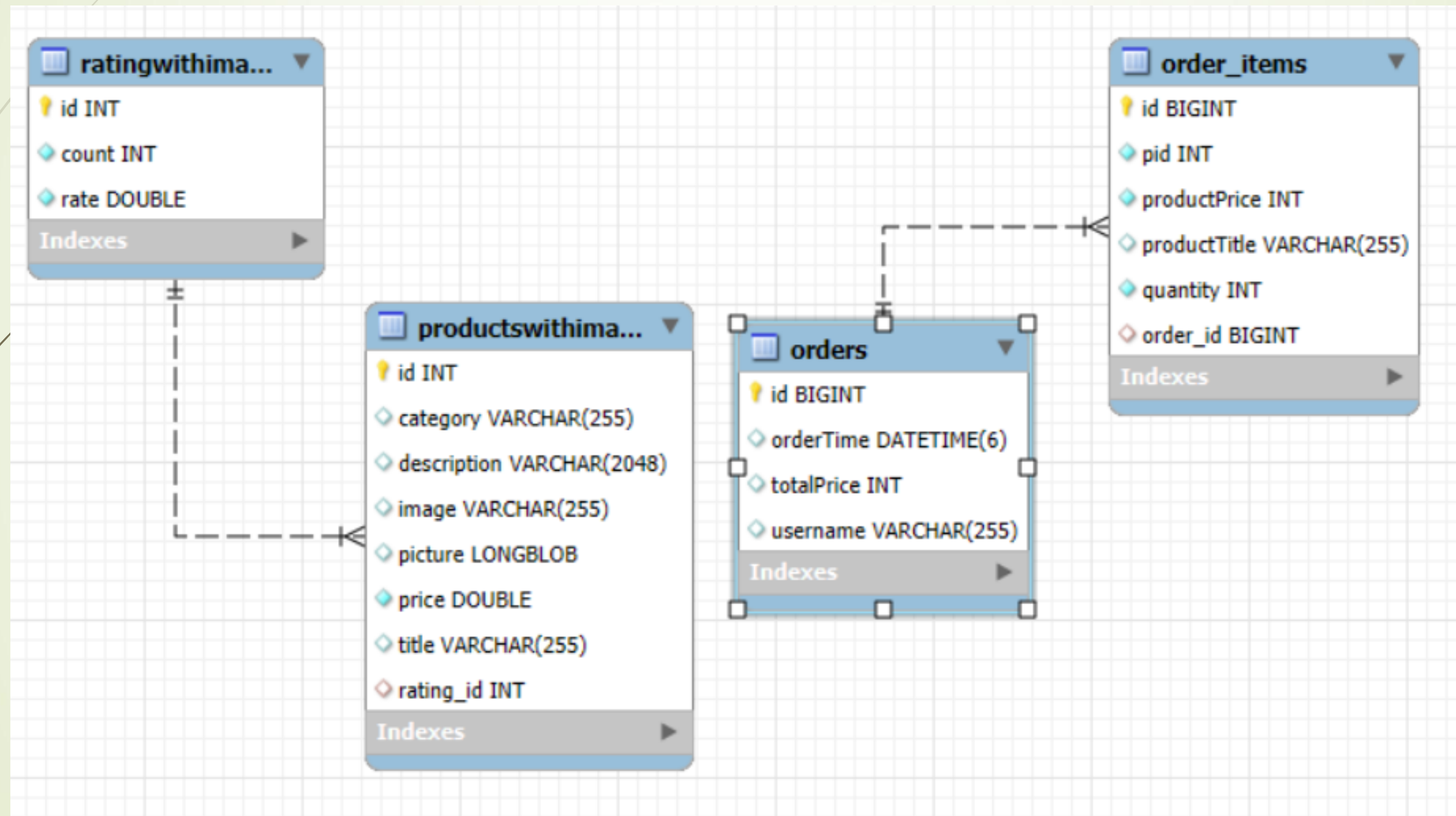
7

White Gold Plated Princess

9

3

資料庫模型圖



資料庫模型圖說明

1. 產品顯示程式只有使用productswithimage表格
並未使用到ratingwithimage表格
2. Order表格和OrderItem表格有關聯

Order表格:

```
@OneToMany(mappedBy = "order",  
cascade = CascadeType.ALL, targetEntity=OrderItem.class)  
private List<OrderItem> items = new ArrayList<>();
```

OrderItem表格:

```
@ManyToOne  
@JoinColumn(name = "order_id")  
private Order order;
```


前端 jQuery 程式

1. 登入功能：驗證用戶輸入的帳號密碼。
2. 商品資料展示
3. 加入購物車功能：點擊產品按鈕後，將商品加入購物車。
4. 送出訂單功能：計算總金額並送出訂單。

API 呼叫流程整理

API 路徑	方法	功能	驗證
/api/user/login	POST	登入取得 JWT	無需 Token
/api/products	GET	取得產品列表	不需要登入
/api/orders	POST	建立新訂單	需要 JWT
/api/orders/{username}	GET	查詢個人訂單	需要 JWT
/api/orders/orderid/{orderid}	GET	查詢訂單資料	需要 JWT
/api/items/{orderid}	GET	查詢訂單商品明細	需要 JWT

前端畫面區塊程式

區塊名稱	說明
登入區塊 (#login)	使用者輸入帳密，點擊登入按鈕，呼叫 <code>/api/user/login</code> 取得 JWT Token
產品列表 (#products)	顯示所有商品，載入 <code>/api/products</code> 資料，可加入購物車
購物車 (#cart)	顯示目前購物車內容，可刪除商品、送出訂單（呼叫 <code>/api/orders</code> ）
訂單管理 (#orders)	顯示登入者的所有訂單 <code>/api/orders/{username}</code> ，可點擊查看每筆訂單商品明細

登入功能

```
$('#loginBtn').click(function (e) {  
  e.preventDefault();  
  const user = $('#username').val();  
  const pass = $('#password').val();  
  
  $.ajax({  
    url: "http://localhost:8080/api/user/login",  
    type: "POST",  
    contentType: "application/json",  
    data: JSON.stringify({ username: user, password: pass }),  
    success: function (res) {  
      localStorage.setItem("token", res.token);  
      console.log("encoded data :" + res.token);  
      isLoggedIn = true;  
      $('#loginMessage').text('');  
      $('#loginStatus').text(`歡迎, ${user}`);  
      sessionStorage.setItem("username", `${user}`);  
      alert("登入成功!");  
    },  
    error: function (xhr) {  
      $('#loginMessage').text('帳號或密碼錯誤');  
    }  
  });  
});
```

Google Yahoo奇摩 登入 - Yahoo奇摩 登入 Facebook

我的商城 帳戶登入 產品列表 訂單 購物車

帳戶登入

admin

....

登入

127.0.0.1:5500 顯示
登入成功！
確定

顯示商品

```
function loadProducts() {
    $('#productList').empty();
    $.ajax({
        url: "http://localhost:8080/api/products",
        type: "GET",
        dataType: "json",
        success: function (products) {
            myproducts = products;
            $.each(products, function (i, product) {
                $('#productList').append(`
                    <div class="col-md-3">
                        <div class="card mb-3">
                            <div class="card-body">
                                <h5 class="card-title">${product.title}</h5>
                                
                                <p class="card-text">價格: ${product.price} 元</p>
                                <p class="card-text">購買數量: <input type="text" id="qty${i}" value="1"></p>
                                <button class="btn btn-success" onclick="addToCart(${product.id},qty${i})">加入購物車</button>
                            </div>
                        </div>
                    </div>
                `);
            });
        },
        error: function (xhr) {
            $('#loginMessage').text('帳號或密碼錯誤');
        }
    });
}
```

我的商城 帳戶登入 產品列表 訂單 購物車

歡迎, mary

產品列表

Fjallraven -
Foldsack No. 1
Backpack, Fits 15
Laptops



價格: 109.95 元

購買數量:

加入購物車

Mens Casual
Premium Slim Fit
T-Shirts



價格: 22.3 元

購買數量:

加入購物車

Mens Cotton
Jacket



價格: 55.99 元

購買數量:

加入購物車

Mens Casual Slim
Fit



價格: 15.99 元

購買數量:

加入購物車

加入購物車

```
function addToCart(productId,qty) {  
  const product = myproducts.find(p => p.id === productId);  
  console.log("qty:"+$(qty).val())  
  const product2={...product,"quantity":$(qty).val()}  
  cart.push(product2);  
  console.log("product:"+JSON.stringify(product2));  
  alert(`已將 ${product2.title} 加入購物車`);  
}
```

登入 - Yahoo奇摩

登入

Facebook

購物車

127.0.0.1:5500 顯示

已將 Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops 加入購物車

確定

產品列表

Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops



價格：109.95 元

購買數量：

加入購物車

M... Fit T-Shirts



價格：22.3 元

購買數量：

加入購物車



價格：55.99 元

購買數量：

加入購物車

購物車更新

```
function updateCart() {
  $('#cartItems').empty();
  let total = 0;

  if (cart.length === 0) {
    $('#cartItems').append('<li class="list-group-item">購物車是空的</li>');
  } else {
    cart.forEach((item, index) => {
      total += item.price*item.quantity;
      $('#cartItems').append(`
        <li class="list-group-item d-flex justify-content-between align-items-center">
          ${item.title} - ${item.price} 元 ,數量: ${item.quantity}
          <button class="btn btn-sm btn-danger" onclick="removeFromCart(${index})">刪除</button>
        </li>
      `);
    });
  }
  $('#totalPrice').text(total);
}
```

我的商城 帳戶登入 產品列表 訂單 購物車

購物車

Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops - 109.95 元 ,數量: 1

刪除

Mens Casual Premium Slim Fit T-Shirts - 22.3 元 ,數量: 1

刪除

John Hardy Women's Legends Naga Gold & Silver Dragon Station Chain Bracelet - 695 元 ,數量: 1

刪除

Solid Gold Petite Micropave - 168 元 ,數量: 1

刪除

總金額 : 995.25 元

送出訂單

submitOrder() - 提交訂單功能

功能：

- .檢查使用者是否已登入
- .組合訂單資料後送出 POST 請求至後端 API

邏輯：

- 1.若未登入，跳出提示訊息。
- 2.取得使用者帳號、計算總金額。
- 3.組成訂單 JSON 結構（包含商品明細）。
- 4.使用 AJAX 將訂單送出，附上 Token 驗證。
- 5.若成功則清空購物車並更新畫面。

```
function submitOrder() {  
  console.log('total:' + total);  
  const order = {  
    username: username,  
    totalPrice: total,  
    items: cart.map(p => ({  
      pid: p.id,  
      productTitle: p.title,  
      productPrice: p.price,  
      quantity: p.quantity  
    })))  
  };  
  console.log("orders list :" + JSON.stringify(order));  
  
  $.ajax({  
    url: "http://localhost:8080/api/orders",  
    type: "POST",  
    contentType: "application/json",  
    headers: {  
      "Authorization": "Bearer " + localStorage.getItem("token")  
    },  
    data: JSON.stringify(order),  
    success: function (data) {  
      alert("訂單已送出！");  
      cart = [];  
      updateCart();  
    }  
  });  
}
```


showOrders() - 顯示訂單清單

- 功能：
- 檢查是否登入
- 從後端取得登入者的訂單清單，顯示訂單資訊與「顯示商品」按鈕

```
$.ajax({
  url: "http://localhost:8080/api/orders/"+sessionStorage.getItem("username"),
  type: "GET",
  dataType: "json",
  success: function (orders) {
    $.each(orders, function (i, order) {
      $('#orderList').append(`
        <div class="col-md-3">
          <p>${order.id}</p>
        </div>
        <div class="col-md-3">
          <p>${order.username}</p>
        </div>
        <div class="col-md-3">
          <p>${order.orderTime}</p>
        </div>
        <div class="col-md-3">
          <button class="btn btn-success" onclick="showDetails(${order.id})">顯示定購商品</button>
        </div>
      `);
    });
  },
  error: function (xhr) {
    $('#loginMessage').text('帳號或密碼錯誤');
  }
});
```


showDetails(orderid) - 顯示特定訂單詳細資料

功能：

顯示訂單基本資訊（ID、使用者、時間）

呼叫 showItemDetails 顯示此筆訂單的商品內容

```
$.ajax({
  url: "http://localhost:8080/api/orders/orderid/"+orderid,
  type: "GET",
  dataType: "json",
  success: function (order) {

    $('#orderList').append(`
      <div class="col-md-4">
        <p>${order.id}</p>
      </div>
      <div class="col-md-4">
        <p>${order.username}</p>
      </div>
      <div class="col-md-4">
        <p>${order.orderTime}</p>
      </div>
    `);

  },
  error: function (xhr) {
    $('#loginMessage').text('帳號或密碼錯誤');
  }
});
```

```
$.ajax({
  url: "http://localhost:8080/api/items/"+orderid,
  type: "GET",
  dataType: "json",
  success: function (items) {
    $.each(items, function (i, item) {
      $('#itemList').append(`
        <div class="col-md-3">
          <p>${item.pid}</p>
        </div>
        <div class="col-md-3">
          <p>${item.productTitle}</p>
        </div>
        <div class="col-md-3">
          <p>${item.productPrice}</p>
        </div>
        <div class="col-md-3">
          <p>${item.quantity}</p>
        </div>
      `);
    });
  },
  error: function (xhr) {
    $('#loginMessage').text('帳號或密碼錯誤');
  }
});
```

Bootstrap應用

1. 使用 Bootstrap 4.x 作為前端框架，提升開發效率。
2. 與 jQuery 功能，建立互動性前端介面。
3. 運用 Row、Card、Button、Form、List Group 等元件設計購物網站。

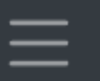
Bootstrap 4.x - nav 元件導覽

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <a class="navbar-brand" href="#">我的商城</a>
  <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#collapsibleNavbar">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="collapsibleNavbar">
    <ul class="navbar-nav">
      <li class="nav-item"><a class="nav-link" href="#" data-target="login">帳戶登入</a></li>
      <li class="nav-item"><a class="nav-link" href="#" data-target="products">產品列表</a></li>
      <li class="nav-item"><a class="nav-link" href="#" data-target="orders">訂單</a></li>
      <li class="nav-item"><a class="nav-link" href="#" data-target="cart">購物車</a></li>
    </ul>
    <span class="ml-auto navbar-text text-white" id="loginStatus">未登入</span>
  </div>
</nav>
```

1. 當視窗縮小時候會顯示縮圖模式



```
<button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#collapsibleNavbar">
  <span class="navbar-toggler-icon"></span>
</button>
```

我的商城 

帳戶登入

登入

Bootstrap - Card 元件應用

- Card 元件顯示商品資訊
 - `<div class='row'> <div class='col-md-3'>` 進行列行排版。
 - `<div class='card'>` 元件展示商品資訊。
 - `<img class='card-img-top'>` 顯示產品圖片。
 - `<button class='btn btn-success'>` 加入購物車按鈕。

Bootstrap - 購物車內容

- 使用 List Group 呈現購物車內容：：`<ul class='list-group'>`。
- 每個商品項目：`<li class='list-group-item d-flex justify-content-between align-items-center'>`。
- 更新總金額顯示：`$('#totalPrice').text(total)`。

Bootstrap - 訂單頁面

- `<h3>訂單管理</h3>`
- `<div id="orderList" class="row"></div>`
- `<h3>商品明細</h3>`
- `<div id="itemList" class="row"></div>`
- 訂單資訊列呈現訂單編號、用戶、時間。
- 商品細節區塊：分欄顯示產品編號、名稱、價格與數量。

後端架構概述

- 使用 Spring Boot 框架建構 RESTful API。
- 控制層(Controller)：處理前端請求。
- 模型層(Model)：定義資料結構。
- 資料存取層(Repository)：連接資料庫。
- 使用 JWT 驗證用戶身份。
- 資料庫：儲存使用者、商品與訂單明細資料。

JWT 驗證與帳號

產生 JWT Token

```
public static String generateToken(String username) {  
    return Jwts.builder()  
        .setSubject(username)  
        .setIssuedAt(new Date())  
        .setExpiration(new Date(System.currentTimeMillis() + 86400000)) // 1天有效  
        .signWith(SignatureAlgorithm.HS512, SECRET)  
        .compact();  
}
```

呼叫此方法可為登入使用者產生 JWT，並設定有效期限為「一天」。

驗證 JWT Token 是否有效

```
public static boolean validateToken(String token) {  
    try {  
        String name = Jwts.parser()  
            .setSigningKey(SECRET)  
            .parseClaimsJws(token)  
            .getBody()  
            .getSubject();  
        return name != null;  
    } catch (Exception e) {  
        System.out.println("validateToken error " + e.getMessage());  
        return false;  
    }  
}
```


UserController功能簡介

- 使用 `@RestController` 和 `@RequestMapping("/api/user")`，定義 REST API 端點 (基於 `/api/user`)。
- `@CrossOrigin`：允許跨域請求(預設允許所有來源)，方便前端(`usercart.html`) 訪問。
- `@Autowired`：注入 `JwtUtility`(用於生成和解析 JWT token)。
- 使用內存中的 `users` 列表模擬用戶資料庫(編碼用戶名和密碼)。
- 使用 `generateToken(String username)`，當使用者登入成功後，系統會產生一個包含帳號名稱、簽發時間與一天有效期限的 JWT Token，並透過 HS512 演算法與預設密鑰進行加密簽章。

登入驗證 API

```
@PostMapping("/login")
```

```
public ResponseEntity<?> login(@RequestBody Map<String, String> payload,  
HttpSession session) {
```

.透過 **POST /api/user/login** 登入，前端應以 JSON 格式送出帳密。

```
{  
    "username": "admin",  
    "password": "1234"  
}
```

.如果密碼符合，回傳 token 給前端，並將使用者名稱存在 session 中。

.錯誤時回傳 401 Unauthorized 錯誤訊息。

```
@PostMapping("/login")  
public ResponseEntity<?> login(@RequestBody Map<String, String> payload, HttpSession session) {  
    String username = payload.get("username");  
    String password = payload.get("password");  
    Map<String, String> u1=users.stream().filter(m->password.equals(m.get(username))).findAny().orElse(null);  
    if (u1!=null) {  
        session.setAttribute("loginname", username);  
        String token = jwtUtil.generateToken(username);  
        System.out.println(username+" 登入成功");  
        return ResponseEntity.ok(Map.of("token", token));  
    } else {  
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)  
            .body(Map.of("message", "帳號或密碼錯誤"));  
    }  
}
```

Product 實體類別

Product 實體類別（對應資料表 productswithimage）

@Data

@Entity

@Table(name="productswithimage")

public class Product

欄位名稱	說明	備註
id	商品主鍵	Primary Key
title	商品名稱	最長 255 字
description	商品描述	最長 2048 字
category	商品分類	如「手機」、「電腦」等
price	商品價格	double 數值
image	圖片路徑或圖片 URL	文字格式儲存
picture	圖片二進位資料	@Lob 支援圖片上傳
rating_id	評分 ID（關聯其他表用）	可用來連接評分表

ProductController功能簡介

- 使用 `@RestController` 和 `@RequestMapping("/api/products")`，定義 REST API 端點(基於 `/api/products`)。
- `@CrossOrigin`：允許跨域請求(預設允許所有來源)，方便前端(`usercart.html`)訪問。
- `@Autowired`：注入 `ProductRepository`(用於資料庫操作，假設使用 Spring Data JPA)。
- 使用 MySQL 資料庫，儲存產品資料(`products` 表)。

Order 實體類別

@Entity

```
@Table(name = "orders")
```

@Data

```
public class Order {
```

@ld

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

```
private String username;
```

```
private LocalDateTime orderTime;
```

```
private Integer totalPrice;
```

```
@OneToMany(mappedBy = "order", cascade = CascadeType.ALL, targetEntity=OrderItem.class)
```

```
private List<OrderItem> items = new ArrayList<>();
```

}

[illegible]

OrderController功能簡介

建立新訂單 **POST /api/orders**

@PostMapping

```
public ResponseEntity<?> createOrder(@RequestBody Order order,  
@RequestHeader("Authorization") String token)
```

✿ 流程說明：

從 JWT token 中解析出登入者帳號：

```
String username = JwtUtil.extractUsername(token.replace("Bearer ", ""));
```

設定訂單時間與關聯每筆明細回主訂單：

```
order.setOrderTime(LocalDateTime.now());
```

```
item.setOrder(order);
```

呼叫 JPA 儲存整張訂單（主檔 + 明細）：

```
Order rs=orderRepo.save(order);
```

儲存後為了避免迴圈依賴，將每筆 item 的 order 欄位設為 null，否則序列化時可能 StackOverflow。

OrderController功能簡介

查詢全部訂單（管理員用）：GET /api/orders

@GetMapping

```
public ResponseEntity<List<Order>> getAllOrders()
```

- 查詢所有訂單。
- 同樣將明細設為 null，只顯示訂單基本資訊。

查詢單筆訂單（含明細）：GET /api/orders/orderid/{orderid}

@GetMapping("/orderid/{orderid}")

```
public ResponseEntity<Order> getOrdersById(@PathVariable("orderid") long  
orderid)
```

- 根據訂單編號查詢單筆訂單。
- 預設會包含訂單明細，如果你想保留明細給前端查閱，這樣設計是可以的。

OrderItem 實體類別

@Entity

```
@Table(name = "order_items")
```

@Data

```
public class OrderItem {
```

@Id

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

```
private int pid;
```

```
private String productTitle;
```

```
private int productPrice;
```

```
private int quantity;
```

@ManyToOne

```
@JoinColumn(name = "order_id")
```

```
private Order order;
```

[illegible]

OrderItemController功能簡介

查詢單筆訂單明細：GET /api/items/{orderid}

@GetMapping("/{orderid}")

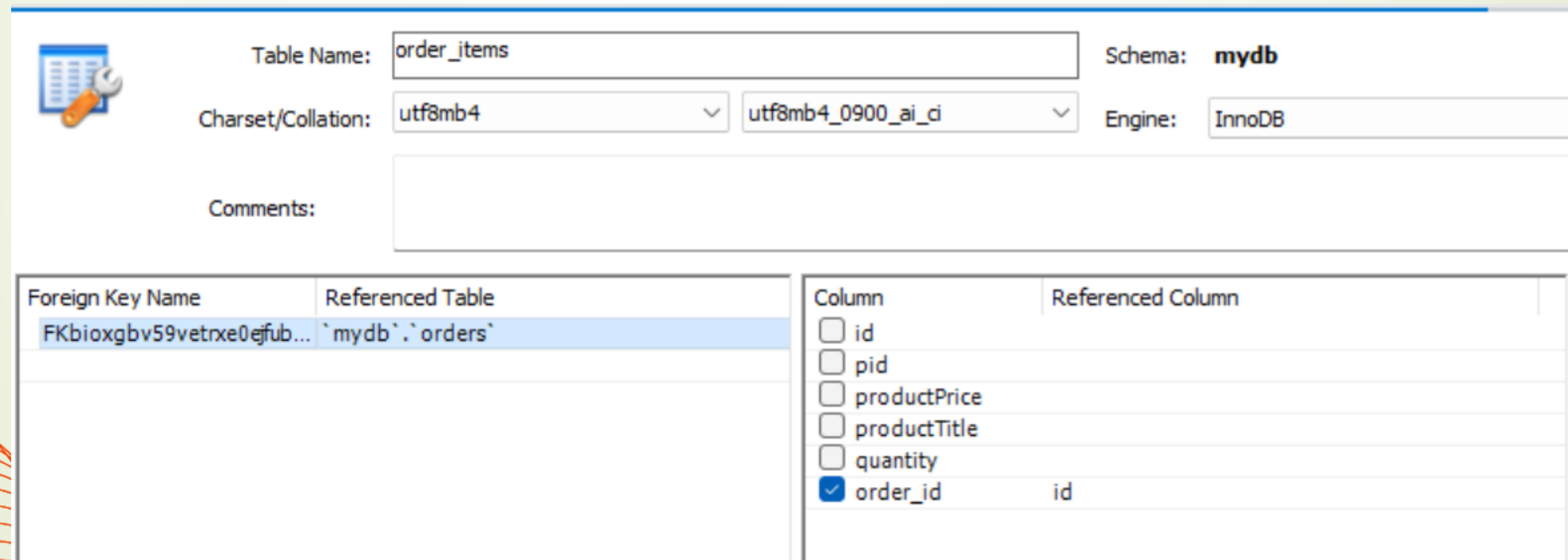
public ResponseEntity<List<OrderItem>> getByOrderId(@PathVariable("orderid")
long orderid)

- 根據訂單編號查詢單筆訂單明細

```
@RestController
@RequestMapping("/api/items")
@CrossOrigin(origins="*")
public class OrderItemController {
    @Autowired
    ItemRepository itemsRepo;
    @GetMapping("/{orderid}")
    public ResponseEntity<List<OrderItem>> getByOrderId(@PathVariable("orderid") long orderid){
        List<OrderItem> items=itemsRepo.findByOrderId(orderid);
        for(OrderItem i : items)
        {
            i.setOrder(null);
        }
        return ResponseEntity.ok(items);
    }
}
```


資料庫Order_items和Orders關聯性

- Order_items是「訂單項目」的資料表，用來記錄每張訂單中包含了哪些商品。
- 每一筆Order_items資料中會包含一個Order_id欄位，指向Orders資料表中的id欄位。



The screenshot shows the MySQL Workbench interface for configuring a foreign key in the `order_items` table. The table name is `order_items`, the schema is `mydb`, and the engine is `InnoDB`. The charset/collation is set to `utf8mb4` with `utf8mb4_0900_ai_ci`. The foreign key is named `FKbioxgbv59vetrxe0ejub...` and references the `orders` table in the `mydb` schema. The columns involved are `order_id` in the `order_items` table and `id` in the `orders` table.

Foreign Key Name	Referenced Table
FKbioxgbv59vetrxe0ejub...	`mydb`.`orders`

Column	Referenced Column
☐ id	
☐ pid	
☐ productPrice	
☐ productTitle	
☐ quantity	
☒ order_id	id

結論

- 本專案，整合前端與後端技術，實作會員登入、商品展示、購物車、訂單管理等功能。前端使用 HTML、JavaScript與 Bootstrap建構互動式介面，並透過 AJAX 串接後端 RESTful API。後端則採用 Spring Boot 搭配 JWT 驗證與資料庫(MySQL)，實現完整的帳號驗證與訂單資料管理，本專案展示了前後端分離商城系統。